

title: "rv32im_core — Hardware IP Specification"
revision: "0.3"
date: "2026-08-29"
status: "Draft — Gate 3b Complete"

Field	Value
IP Name	rv32im_core
Revision	0.3 (structured_spec rev 0.4)
Date	2026-08-29
Status	Draft — Gate 3b Complete
Gate Status	Gate 1 ✓ · Gate 2 ✓ · Gate 3 ✓
Tool Flow	Claude Code RTL Gen Flow
Note: Testbench (Phase 4) and Verification (Phase 5) have not been run. Verification Summary (Ch. 07) is omitted. Synthesis data (Ch. 08) is from RTL gate-level with GF180MCU PDK.	

Quick Reference

Module Hierarchy

```
rv32im_core (top — purely structural, instantiates I103–I118)
├── rv32im_if_stage — Instruction Fetch (PC, Fetch FSM, I-Bus)
│   └── (2-state FSM: ST_FETCH → ST_WAIT)
├── rv32im_id_stage — Instruction Decode
│   └── rv32im_decoder — Opcode → control signals (11 enum types)
│       └── rv32im_imm_gen — Immediate gen (I/S/R/U/W)
├── rv32im_regfile — 32x32 register file, write-first bypass
├── rv32im_ex_stage — Execute
│   ├── rv32im_alu — RV32I arithmetic/logic/shift/compare
│   ├── rv32im_muldiv — RV32M multiply/divide (PR_M_EXT_EN=1)
│   │   ├── rv32im_mult — Multiplier (single-cycle)
│   │   └── rv32im_div — Divider (iterative, non-restoring, PR_DIV_IMPL=0)
│   └── rv32im_branch_unit — Branch condition + target address
├── rv32im_csr_file — 16 M-mode CSRs (PR_CSR_EN=1)
├── rv32im_mem_stage — Memory Access
│   └── rv32im_lew — Load/Store byte-enable alignment
├── rv32im_trap_ctrl — Trap & interrupt controller (PR_IRQ_EN=1)
└── rv32im_hazard_ctrl — Hazard detection, stall, forwarding
```

Package rv32im_pkg defines 11 enum types and 4 pipeline bundle structs (ifid_t, idex_t, exmem_t, memwb_t).

Parameter Summary

Parameter	Type	Default	Description
PR_BOOT_ADDR	logic [31:0]	32'h8000_0000	PC reset / boot address (must
PR_MTVEC_RESET	logic [31:0]	32'h0000_0000	mtvec reset value (must be wor
PR_HART_ID	logic [31:0]	32'h0	Hardware thread ID (mhartid CS
PR_M_EXT_EN	bit	1	Enable RV32M extension
PR_MULT_IMPL	int unsigned	0	Multiplier impl (0=single-cycl
PR_DIV_IMPL	int unsigned	0	Divider impl (0=SEQ non-restor
PR_CSR_EN	bit	1	Enable CSR register file
PR_IRQ_EN	bit	1	Enable interrupt/trap (require
PR_COUNTER_EN	bit	1	Enable mcycle/minstret (requir
PR_MTVEC_VEC_EN	bit	0	Vectored trap mode (0=direct)
PR_FWD_EN	bit	1	Enable EX/MEM→EX forwarding
PR_RF_RESET_EN	bit	1	Reset register file on reset
PR_RF_IMPL	int unsigned	0	Reg-file impl (0=FF; 1=LUTRAM,
PR_TRACE_EN	bit	0	Enable commit trace port
PR_BUS_OUTSTANDING	int unsigned	1	Max in-flight bus requests (lo

Feature Matrix

Feature	Description	Enabled	Notes
F01	RV32I base ISA (40 instructions) ✓		§1.1.1 of spec
F02	RV32M: MUL/MULH/MULHSU/MULHU/D		PR_M_EXT_EN=1
F03	Zicsr: CSRRW/S/C, CSRRWI/SI/CI ✓		PR_CSR_EN=1
F04	Zifencei: FENCE.I — flush + re ✓		—
F05	5-stage in-order pipeline, sin ✓		Core architecture
F06	Full forwarding EX/MEM→EX and ✓		PR_FWD_EN=1
F07	Load-use interlock (stall on c ✓		—
F08	Branch resolve at EX, predict- ✓		2-cycle branch penalty
F09	Regfile 32×32, x0=0, write-fir ✓		PR_RF_IMPL=0
F10	Harvard bus (I-Bus + D-Bus), v ✓		—
F11	M-mode CSR file — 16 CSRs ✓		PR_CSR_EN=1
F12	Precise exceptions (commit at ✓		PR_IRQ_EN=1
F13	9 synchronous exceptions ✓		ECALL/EBREAK/illegal/misalign/
F14	3 M-mode interrupts (SW/Timer/ ✓		PR_IRQ_EN=1
F15	MRET restores mstatus.MIE and ✓		—
F16	mcycle / minstret 64-bit count ✓		PR_COUNTER_EN=1
F17	MULDIV multi-cycle stall, bit- ✓		PR_M_EXT_EN=1
F18	Retire trace port ✗		PR_TRACE_EN=0
F19	mtvec Vectored mode ✗		PR_MTVEC_VEC_EN=0

01 — Overview

1.1 Introduction

rv32im_core is a 32-bit RISC-V machine-mode processor core implementing the RV32IM instruction set architecture. It implements a 5-stage in-order scalar pipeline (IF → ID → EX → MEM → WB) with full data hazard handling (forwarding + stall) and precise exception/interrupt support.

The core provides a Harvard-style memory interface: a separate instruction bus (I-Bus, read-only) and data bus (D-Bus, read/write), both using a valid/ready handshake protocol. The top module (rv32im_core) is purely structural — it instantiates 16 sub-modules and connects them via typed pipeline bundle signals.

This document corresponds to spec revision 0.4 (structured_spec.json) / design revision 0.3 (final_config.json), with Gate 3b complete (RTL generated, linted, synthesised, and SVA properties reviewed offline). Testbench generation and verification are pending.

1.2 Key Features

- RV32I: 40 instructions (37 base + FENCE + ECALL + EBREAK; FENCE.I via Zifencei)
- RV32M: MUL, MULH, MULHSU, MULHU, DIV, DIVU, REM, REMU (multi-cycle iterative divider)
- 5-stage in-order pipeline, single-issue, static predict-not-taken
- Load-use stall (1-cycle penalty, detected in ID)
- Dual forwarding paths: EX/MEM→EX, MEM/WB→EX (PR_FWD_EN=1)
- 2-cycle branch penalty (resolved at EX)
- Machine-mode CSR: 16 registers per §16.4 of spec
- 9 synchronous exceptions + 3 asynchronous interrupts (SW, Timer, External)
- Commit-point at MEM stage (precise exceptions)
- 64-bit mcycle and minstret counters
- Configurable boot address (PR_BOOT_ADDR)
- Synthesised to GF180MCU: 12,731 cells, 2,219 flip-flops, 371,272 μm² (TT 025C 1.8V)

1.3 Top-Level Parameters

See Quick Reference → Parameter Summary.

1.4 Module Hierarchy

See Quick Reference → Module Hierarchy.

02 — Port List

Clock & Reset

Port	Direction	Width	Description
i_clk_core	input	1	Core clock; all FFs on rising
i_resetrn_core	input	1	Active-low synchronous reset

*Per REQ-028: i_resetrn_core is active-low, synchronous assert, synchronous deassert.
The core does NOT instantiate synchroniser chains — caller is responsible for clean reset.*

Instruction Bus (I-Bus, read-only)

Port	Direction	Width	Description
o_imem_req_valid	output	1	Fetch request valid
i_imem_req_ready	input	1	Fetch request accepted
o_imem_req_addr	output	32	Fetch address (word-aligned)
i_imem_rsp_valid	input	1	Instruction word valid (1-cycl
i_imem_rsp_rdata	input	32	Instruction word
i_imem_rsp_err	input	1	Instruction fetch bus error

Data Bus (D-Bus, read/write)

Port	Direction	Width	Description
o_dmem_req_valid	output	1	Data request valid
i_dmem_req_ready	input	1	Data request accepted
o_dmem_req_addr	output	32	Data address
o_dmem_req_we	output	1	Write enable (1=write, 0=read)
o_dmem_req_be	output	4	Byte enable (one bit per byte)
o_dmem_req_wdata	output	32	Write data
i_dmem_rsp_valid	input	1	Data response valid (1-cycle)
i_dmem_rsp_rdata	input	32	Read data
i_dmem_rsp_err	input	1	Data bus error

Interrupt

Port	Direction	Width	Description
i_irq_sw	input	1	Software interrupt (MSIP), lev
i_irq_timer	input	1	Timer interrupt (MTIP), level-
i_irq_ext	input	1	External interrupt (MEIP), lev

Per REQ-031: Interrupt signals are level-sensitive. The core does NOT include input synchronisers — caller must synchronise before assertion.

Trace / Debug (PR_TRACE_EN=0 — ports exist but are tied off)

Port	Direction	Width	Description
o_trace_valid	output	1	Instruction committed this cyc
o_trace_pc	output	32	PC of committed instruction
o_trace_instr	output	32	Raw instruction word
o_trace_rd_wen	output	1	Register-file write enable
o_trace_rd_addr	output	5	Destination register index
o_trace_rd_wdata	output	32	Value written to register

03 — Clock & Reset

Clock Domain

Domain	Signal	Description
clk0	i_clk_core	Single clock domain — all FFs

No multi-clock-domain crossings. No asynchronous paths.

Reset Strategy

- Type: Active-low synchronous (i_resetrn_core)
- Assert: Held low for ≥ 1 clock cycle
- Effect: All pipeline registers, CSR state, PC, hazard state cleared
- Register file: Reset to zero (PR_RF_RESET_EN=1)

Boot Sequence

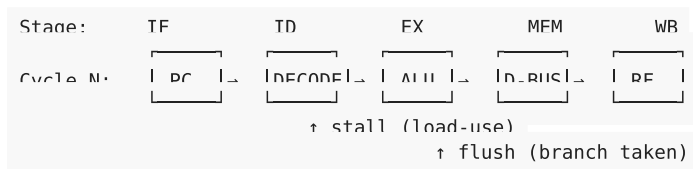
On reset de-assertion (first rising edge with i_resetrn_core=1):

- $PC \leftarrow PR_BOOT_ADDR = 32'h8000_0000$
- I-Bus begins fetching at address 0x8000_0000 on next cycle
- CSR reset values: mstatus=0 (MIE=0), mtvec=PR_MTVEC_RESET=0, mip=0, mepc=0
- Pipeline fills over 4 cycles before first instruction retires

Per REQ-020/021: PR_BOOT_ADDR[1:0] and PR_MTVEC_RESET[1:0] must be 2'b00 (word-aligned).

04 — Microarchitecture

Pipeline Overview



- Issue width: 1 (scalar)
- Execution model: In-order; no OOO, no branch prediction beyond predict-not-taken
- Commit point: MEM stage (precise exceptions guaranteed)
- Pipeline registers: ifid_t, idex_t, exmem_t, memwb_t (typed structs from rv32im_pkg)

Stage Description

Stage	Module	Function
IF	rv32im_if_stage	PC register, 2-state Fetch FSM
ID	rv32im_id_stage	Decoder, immediate gen, regist
EX	rv32im_ex_stage	ALU, MUL-DIV, branch resolve,
MEM	rv32im_mem_stage	D-Bus handshake, byte-enable,
WB	(in rv32im_core)	Register file write-back

Supporting modules:

Module	Function
rv32im_hazard_ctrl	Load-use stall detect, forward
rv32im_trap_ctrl	Exception/interrupt arbitratio
rv32im_csr_file	16 M-mode CSR registers

Instruction Fetch FSM (IF Stage)

The fetch FSM has 2 states (per REQ-048):

- ST_FETCH: Assert o_imem_req_valid, present next PC, wait for i_imem_req_ready
- ST_WAIT: Request accepted, wait for i_imem_rsp_valid

Redirect during ST_WAIT (branch taken, trap): must still consume the in-flight response (per REQ-050, bus rule B6), then restart fetch from the new PC.

Latency-0 response (per REQ-049): reg_rsp_early captures i_imem_rsp_valid at the

handshake cycle itself, allowing zero-cycle memory to be supported.

PC Priority Mux (per REQ-047)

Priority (highest first):

1. Tran/MRET redirect from MEM stage (synchronous exception, MRET)
2. Branch taken redirect from EX stage
3. PC+4 (sequential fetch)

Hazard Handling

Load-use stall (REQ-007):

- Detected in ID when EX instruction is a load and rs1/rs2 of incoming ID instruction match the load's rd
- Action: stall IF and ID for one cycle, inject NOP bubble into EX

Forwarding (REQ-006, PR_FWD_EN=1):

- EX/MEM → EX: result of instruction N-1 forwarded to EX operand of instruction N
- MEM/WB → EX: result of instruction N-2 forwarded (covers case where register write hasn't retired)
- Regfile write-first bypass (REQ-009): simultaneous read/write to same register returns new value

Branch penalty (REQ-008):

- Branch condition evaluated at end of EX
- On taken: flush IF and ID (2-cycle penalty), redirect PC to branch target
- Predict-not-taken: instructions in IF/ID on a not-taken branch are already correct

Multiply/Divide (REQ-017)

rv32im_muldiv stalls the EX stage while the operation completes:

- Multiplier (PR_MULT_IMPL=0): Single-cycle result
- Divider (PR_DIV_IMPL=0): Iterative non-restoring, variable latency (up to 34 cycles for 32-bit)
- Special cases handled bit-exactly per RISC-V spec (divide-by-zero, overflow)

05 — CSR / Register Map

(Active: PR_CSR_EN=1)

Implemented CSRs (16 registers per §16.4)

Address	Name	Access	Reset	Description
0x300	mstatus	RW	0x0000_0000	MIE, MPIE, MPP fields
0x304	mie	RW	0x0000_0000	MSIE, MTIE, MEIE enable bits
0x305	mtvec	RW	PR_MTVEC_RESET	Trap vector (MODE=0 direct; MO
0x310	mstatush	RW	0x0000_0000	mstatus upper (RV32 only)
0x320	mcountinhibit	RW	0x0000_0000	Counter inhibit
0x340	mscratch	RW	0x0000_0000	Scratch for trap handler
0x341	mepc	RW	0x0000_0000	Exception PC (mepc[1:0]=0 forc
0x342	mcause	RW	0x0000_0000	Trap cause (bit[31]: interrupt
0x343	mtval	RW	0x0000_0000	Trap value (fault addr or inst
0x344	mip	RO	—	Pending interrupts (reflects I
0xF11	mvendorid	RO	0x0000_0000	Vendor ID
0xF12	marchid	RO	0x0000_0000	Architecture ID
0xF13	mimpid	RO	0x0000_0000	Implementation ID
0xF14	mhartid	RO	PR_HART_ID	Hardware thread ID
0xB00	mcycle	RW	0x0000_0000	Cycle counter lower 32 bits
0xB80	mcycleh	RW	0x0000_0000	Cycle counter upper 32 bits
0xB02	minstret	RW	0x0000_0000	Retired instruction counter, l
0xB82	minstreth	RW	0x0000_0000	Retired instruction counter, h

mip is read-only and directly reflects the synchronised IRQ inputs.

Vectored Mode

PR_MTVEC_VEC_EN=0 in this configuration — trap mode is direct (MODE=0). All exceptions and interrupts jump to mtvec.BASE.

06 — Functional Description

6.1 Instruction Fetch (IF)

The IF stage manages the instruction bus using a 2-state FSM (ST_FETCH / ST_WAIT). On reset, $PC \leftarrow PR_BOOT_ADDR$. On each fetch cycle, `o_imem_req_valid` is asserted and `o_imem_req_addr` presents the current PC. The stage waits for the request handshake (`i_imem_req_ready`), then waits for `i_imem_rsp_valid` to capture the instruction word.

Latency-0 memory (response on same cycle as request) is supported via `reg_rsp_early`. In-flight responses during a redirect are consumed and discarded (bus rule B6 compliance).

6.2 Instruction Decode (ID)

`rv32im_decoder` decodes the 32-bit instruction into 11 typed control fields (`alu_op`, `op_a_sel`, `op_b_sel`, `imm_sel`, `br_op`, etc.). `rv32im_imm_gen` extracts and sign-extends the immediate per format (I/S/B/U/J). `rv32im_regfile` provides two concurrent reads (`rs1`, `rs2`) with write-first bypass for the same-cycle write case.

Load-use hazard detection runs in ID: if the EX stage holds a pending load and its `rd` matches `rs1` or `rs2` of the incoming instruction, a stall is inserted.

6.3 Execute (EX)

Forwarded operands are multiplexed in at the start of EX. For RV32I operations, `rv32im_alu` computes the result in one cycle. For RV32M operations, `rv32im_muldiv` computes the result (possibly multi-cycle for division) while the pipeline is held.

`rv32im_branch_unit` evaluates the branch condition and computes the target address. On a taken branch, a redirect signal is sent to IF with the target address, and IF/ID pipeline registers are flushed.

CSR instructions (CSRRW/S/C variants) are handled by `rv32im_csr_file`: the old CSR value is read as the EX result, and the write is applied atomically.

6.4 Memory Access (MEM)

For load/store instructions, `rv32im_lsu` issues a D-Bus request (`o_dmem_req_valid`) and waits for acknowledgement. Byte enables are computed from the address alignment and access width (B/H/W). On load response, the data is sign- or zero-extended: LB/LBU (byte), LH/LHU (halfword), LW (word).

Bus errors from `i_imem_rsp_err` or `i_dmem_rsp_err` are forwarded to `rv32im_trap_ctrl` as instruction/load-store access faults.

Exceptions from earlier stages are prioritised and committed at MEM (precise exception semantics). `rv32im_trap_ctrl` captures `mepc`, `mcause`, `mtval`, and generates the trap redirect to IF.

6.5 Write-Back (WB)

The WB stage (implemented in the top-level core) writes the selected result (ALU, load data, or CSR read value) to `rv32im_regfile`. Writes to `x0` are suppressed. The `minstret` counter is incremented on each instruction that reaches WB without being squashed.

6.6 Hazard Control

`rv32im_hazard_ctrl` provides:

- Stall generation (load-use): holds IF/ID pipeline registers, injects NOP bubble
- Forwarding select: two MUX select outputs (one per source operand) indicating whether to use the EX-stage result, MEM-stage result, or register-file output

6.7 Trap & Interrupt Handling (PR_IRQ_EN=1)

`rv32im_trap_ctrl` arbitrates between:

- Synchronous exceptions (9 types): illegal instruction, ECALL, EBREAK, instruction misalign, instruction access fault, load/store misalign, load/store access fault
- Asynchronous interrupts (3 types): MSIP (software), MTIP (timer), MEIP (external)

Synchronous exceptions take priority over interrupts. Within synchronous exceptions, earlier pipeline stages have higher priority (precise model).

Trap entry:

- mepc $\leftarrow$ PC of faulting instruction (or next sequential PC for interrupts)
- mcause $\leftarrow$ cause code
- mtval $\leftarrow$ fault address (load/store/fetch) or faulting instruction (illegal)
- mstatus.MPIE $\leftarrow$ mstatus.MIE; mstatus.MIE $\leftarrow$ 0
- PC redirect $\leftarrow$ mtvec.BASE

MRET:

- mstatus.MIE $\leftarrow$ mstatus.MPIE; mstatus.MPIE $\leftarrow$ 1; mstatus.MPP $\leftarrow$ 0
- PC redirect $\leftarrow$ mepc

07 — Verification Summary

Status: PENDING — Testbench (Phase 4) and simulation (Phase 5) have not been executed for this design snapshot.

>

Gate 3b was approved after offline review of 181 SVA properties. 83 properties were noted as vacuous under the 113-cycle smoke test (antecedent never triggered). The property intent was confirmed correct; stimulus generation is required to exercise them.

>

To generate the testbench: run /tb_generator from the project root.
To run verification: run /verification after testbench generation and RTL are available.

Metric	Status
----- -----	
SVA compile	PASS (Gate 3b)
SVA properties	181 (83 vacuous under smoke te
Simulation	Not run
Mutation testing	Not run
Sign-off	0 / 191 REQ-IDs

08 — Synthesis Report

Summary (GF180MCU TT 025C 1.8V)

Metric	TT 025C 1v80	SS 125C 1v62
----- ----- -----		
PDK	GF180MCU	GF180MCU
Liberty	Full (not stub)	Full (not stub)
Total Cells	12,731	12,731
Sequential (FFs)	2,219	2,219
Wire Count	10,470	10,470
Area Estimate	371,272 μm^2	371,272 μm^2
Synthesis Status	PASS	PASS

Frontend: read_slang Yosys plugin (required because module X import pkg::*; syntax is not parsed by stock read_verilog).

Lint check: Verilator 5.041 -Wall $\rightarrow$ PASS, 0 warnings

Elaboration check: VCS X-2025.06 $\rightarrow$ PASS, 0 errors, 0 warnings

Tool Versions

Tool	Version
----- -----	
Yosys	0.58+35 (git sha1 89f32a415)
VCS	X-2025.06
Verilator	5.041
PDK	GF180MCU (globalfoundries-pdk-
Liberty TT	gf180mcu_fd_sc_mcu7t5v0_tt_02

09 — Requirements Traceability Matrix (RTM)

Total: 191 REQ-IDs · 18 RTL modules tagged · 181 SVA properties · Gate 3b approved
Sign-off: 0 (requires Gate 5 — verification pending)
Source: schemas/rtm.json (191 entries)

Summary by Category

Category	Count	RTL Traced	SVA Traced	TC Traced	Signed Off
Functional	156	156	(partial)	0	0
Interface	18	18	(partial)	0	0
Constraint	9	9	(partial)	0	0
Timing	8	8	(partial)	0	0
Total	191	191		0	0

Feature-Level REQ Mapping (Top-Level)

Feature	REQ-ID	Description	RTL Module(s)
F01	REQ-001	RV32I base ISA (40 instruction	rv32im_alu, rv32im_decoder, rv
F02	REQ-002	RV32M: MUL/MULH/MULHSU/MULHU	rv32im_decoder, rv32im_div, rv
F03	REQ-003	Zicshr: CSRRW/S/C, CSRRWI/SI/CI	rv32im_csr_file, rv32im_decode
F04	REQ-004	Zifencei: FENCE.I — flush pipe	rv32im_if_stage, rv32im_decode
F05	REQ-005	5-stage in-order pipeline, sin	rv32im_core (top)
F06	REQ-006	Full forwarding EX/MEM→EX and	rv32im_hazard_ctrl, rv32im_ex
F07	REQ-007	Load-use interlock — stall whe	rv32im_hazard_ctrl
F08	REQ-008	Branch resolve at EX, static p	rv32im_branch_unit, rv32im_if
F09	REQ-009	Regfile 32×32, x0=0, write-fir	rv32im_regfile
F10	REQ-010	Harvard I-Bus/D-Bus, valid/rea	rv32im_if_stage, rv32im_mem_st
F11	REQ-011	M-mode CSR file — 16 CSRs per	rv32im_csr_file
F12	REQ-012	Precise exceptions, commit at	rv32im_trap_ctrl
F13	REQ-013	9 synchronous exceptions	rv32im_trap_ctrl, rv32im_decod
F14	REQ-014	3 M-mode interrupts (SW/Timer/	rv32im_trap_ctrl
F15	REQ-015	MRET restores mstatus.MIE and	rv32im_trap_ctrl, rv32im_csr_f
F16	REQ-016	mcycle/minstret 64-bit perform	rv32im_csr_file
F17	REQ-017	MULDIV multi-cycle stall, spec	rv32im_muldiv, rv32im_mult, rv
F18	REQ-018	Retire trace port (PR_TRACE_EN	rv32im_core (top)
F19	REQ-019	mtvec Vectored mode (PR_MTVEC	rv32im_trap_ctrl, rv32im_csr_f

Detailed REQ Groups (Interface — 18 REQs)

REQ-ID	Description (condensed)
REQ-028	Clock/reset port group: i_clk_
REQ-029	I-Bus port group: req_valid/ad
REQ-030	D-Bus port group: req_valid/ad
REQ-031	IRQ port group: level-sensitiv
REQ-032	Trace port group: 6× o_trace_*
REQ-033	Bus rule B1: transfer on risin
REQ-034	Bus rule B2: payload stable un
REQ-035	Bus rule B3: req_valid must NO
REQ-036	Bus rule B4: req_ready MAY com
REQ-037	Bus rule B5: max PR_BUS_OUTSTA
REQ-038	Bus rule B6: each accepted req
REQ-039	Bus rule B7: responses in-orde
REQ-040	Bus rule B8: core always ready
REQ-041	Bus rule B9: rsp_valid may occ
REQ-042	Bus rule B10: rsp_rdata is don

REQ-043	rv32im_pkg defines 11 enum typ
REQ-044	Pipeline struct typedefs: ifid
REQ-045	rv32im_core top is purely stru

Constraint REQs (9 REQs)

REQ-ID	Description
REQ-020	C1: PR_BOOT_ADDR[1:0] == 2'b00
REQ-021	C2: PR_MTVEC_RESET[1:0] == 2'b
REQ-022	C3: PR_IRQ_EN=1 requires PR_CS
REQ-023	C4: PR_MTVEC_VEC_EN=1 requires
REQ-024	C5: PR_COUNTER_EN=1 requires P
REQ-025	C6: PR_DIV_IMPL=0 only (SEQ no
REQ-026	C7: PR_BUS_OUTSTANDING=1 locke
REQ-027	C8: PR_RF_IMPL=1 (LUTRAM) only
REQ-028	(see Interface above)
Detailed listing of all 191 REQ-IDs is available in schemas/rtm.json. REQ-046 through REQ-191 cover module-level micro-requirements (fetch FSM state machine, forwarding mux select, CSR field encoding, exception priority tables, divider algorithm, etc.) and are traced to their respective RTL modules and SVA files in the JSON artifact.	

Document generated by Claude Code RTL Gen Flow — spec_pdf_generator Phase 6.
Status: Gate 3b complete. Gate 4 (testbench) and Gate 5 (verification) pending.
Source artifacts: schemas/final_config.json, schemas/rtm.json,
schemas/synth_report.json, src/rtl/filelist.f, src/rtl/rv32im_core.sv